青居智选

基于 API 与轻量级 RAG 的租房合同审查系统

文档类型	技术开发文档
版本	v1.6
适用阶段	MVP 开发期
前端形态	微信小程序（UniApp 框架，HBuilderX 开发 ）

版本变更摘要（**v1.6**）
新增多图上传支持、多轮对话接口（POST /api/chat）及 chat_tool.py 模块；图像标注改为前端文本高亮方案；更新数据流、文件目录、函数签名契约及八周路线图。

部署前置条件（验收硬性要求）

微信小程序体验版及正式版均强制要求后端服务满足以下两项条件，缺一不可：

（1）后端接口必须通过 HTTPS 协议对外提供服务；

（2）必须持有已完成 ICP 备案的域名，并在微信公众平台「开发设置 → 服务器域名」中将该域名配置为 request 合法域名。

推荐方案：阿里云或腾讯云学生机（约 9 元/月）+ 低价域名（5–30 元/年）+ Let's Encrypt 免费 SSL 证书。

ICP 备案审核周期为 7–20 个工作日，须由队友 A 于项目第一天提交申请，不得延误。

一、系统概述

本系统面向在校大学生，提供租房合同的自动化法律风险审查服务。用户通过微信小程序一次性上传合同全部页面的照片，系统完成多图文字提取、法条检索与条款分析，以结构化报告的形式标注风险条款、给出处理建议，并支持基于分析结果的多轮追问对话。

系统定位：不提供正式法律咨询，不替代律师意见，作为大学生签约前的辅助参考与交互式答疑工具。

1.1 技术栈

模块	技术选型	说明
视觉识别	阿里 Qwen-VL-Max API	合同图片转纯文本；Prompt 内置合同鉴别逻辑，非合同图片在此阶段拦截
法律推理	DeepSeek V3 API	低成本强推理，强制 JSON 输出，单次调用成本约 0.03 元
向量检索	LlamaIndex + ChromaDB	RAG 标准搭配，本地持久化，无外部服务依赖
Embedding 模型	BAAI/bge-small-zh-v1.5	中文向量模型，完全免费，首次运行时自动下载（约 100 MB）
缓存	SQLite（Python 标准库）	MD5 文本哈希查重，命中时零成本、毫秒级响应，无新增依赖
后端	Python + FastAPI	异步框架，自动生成接口文档，负责 serve 前端静态文件
前端	UniApp + HBuilderX	微信小程序形态，一套代码编译至小程序，HBuilderX 为官方 IDE；支持多图选择与多轮对话 UI

1.2 前端技术说明

UniApp 为前端框架（基于 Vue.js），HBuilderX 为对应 IDE，二者配合使用，最终编译产物为微信小程序包。两者不存在选择关系，均为必要工具。

前端开发过程中需注意以下微信小程序环境限制：

— 网络请求须使用 uni.request()，不可使用浏览器原生 fetch 或 XMLHttpRequest；
— 多图选取通过 uni.chooseImage() 的 count 参数控制上限（建议设为 10），每张图片经 uni.getFileSystemManager().readFile() 转换为 base64 字符串后，以数组形式传输；
— 多轮对话的历史记录维护在前端 data 中，每次追问时将完整历史随请求一并发送；
— 本地调试阶段在微信开发者工具中开启「不校验合法域名」，上线前须关闭并配置正式域名。

二、数据流

2.1 合同审查流程（POST /api/analyze）

用户一次性上传合同全部页面后，后端按以下顺序执行处理：

```
POST /api/analyze  ←  { images: [base64_p1, base64_p2, ...] }

Step 1  对 images 数组中每张图片循环调用 vision_tool.extract_contract()
        → 任意一张 is_contract: false  →  返回错误响应，终止流程
        → 全部 is_contract: true        →  各页文本按顺序拼接为 contract_text

Step 2  cache_tool.get_cache(contract_text)
        计算 MD5，查询 SQLite
        → 命中  →  直接返回历史 JSON，终止流程
        → 未命中 →  继续

Step 3  rag_tool.retrieve_legal_context(contract_text)
        双路检索：语义向量 Top-3 + 关键词强制召回
        → 输出法条上下文字符串

Step 4  reviewer.review_contract(contract_text, legal_context)
        组装 Prompt → 调用 DeepSeek V3 API
        → 解析并验证 JSON 结构

Step 5  cache_tool.set_cache(contract_text, result)
        写入 SQLite 缓存

        →  返回 JSON 响应至前端
```

2.2 多轮对话流程（POST /api/chat）

审查结果返回后，用户可在对话框中追问，后端将原始分析结果作为上下文注入 Prompt：

```
POST /api/chat  ←  { question: str, context: dict, history: list }

        context   = 第一次 /api/analyze 返回的完整 JSON
        history   = [ {role, content}, ... ]  前端维护，每轮追加

Step 1  chat_tool.build_chat_prompt(question, context, history)
        将 context 序列化后注入系统 Prompt
        拼接 history 构建多轮消息列表

Step 2  调用 DeepSeek V3 API（非结构化自由回复）

        →  返回 { answer: str } 至前端
        →  前端将本轮问答追加至 history
```

2.3 审查接口响应结构

DeepSeek 返回结果经 json.loads() 验证后，原样透传至前端。original_text 字段同时用于前端文本高亮（匹配后以黄色背景标注原句，替代图像标注方案）：

```
{
  "overall_risk": "高风险",
  "summary": "发现 2 处风险条款，建议签署前与房东协商修改。",
  "analysis_results": [
    {
      "clause_id": 1,
      "original_text": "租客若提前退租，押金概不退还",
      "risk_level": "高风险",
      "risk_type": "霸王条款",
      "reason": "押金具有担保性质，房东仅可扣除实际损失，不得全额没收。",
      "legal_basis": "《民法典》第七百零四条",
      "advice": "建议改为：提前 30 天书面通知，押金扣除实际损耗后退还。"
    }
  ]
}
```

字段约束：risk_level 取值为「高风险」「中风险」「低风险」三种；risk_type 为自由文本；legal_basis 格式固定为「《法规名称》第X条」。

文本高亮说明：前端在渲染合同全文时，对 original_text 字段的内容进行字符串匹配，命中文本以高亮样式呈现，用户可直接在合同文本中定位风险条款，无需人工逐行对照卡片。

三、文件结构与模块说明

3.1 后端目录

```
rent_agent_backend/
├── main.py                    # FastAPI 入口
├── requirements.txt           # 依赖版本锁定
├── .env                       # API 密钥（不提交版本控制）
├── cache.db                   # SQLite 缓存文件（自动创建）
├── data/
│   ├── civil_code.txt         # 《民法典》合同编 629-745 条
│   └── rental_regulations.txt # 《城市房屋租赁管理办法》
├── vector_store/              # ChromaDB 持久化目录（自动生成）
├── scripts/
│   └── build_index.py         # 向量库构建脚本（仅运行一次）
└── core/
    ├── vision_tool.py         # Qwen-VL 调用（支持多图循环）
    ├── cache_tool.py          # SQLite 读写
    ├── rag_tool.py            # 双路法条检索
    ├── reviewer.py            # Prompt 组装与 DeepSeek 调用
    └── chat_tool.py           # 多轮对话 Prompt 构建
```

3.2 前端目录

```
rent_agent_frontend/           # HBuilderX 打开此目录
├── pages/
│   ├── index/
│   │   └── index.vue          # 主页面：多图上传、审查结果渲染、文本高亮
│   └── chat/
│       └── chat.vue           # 对话页面：追问输入、历史记录展示
├── static/                    # 静态资源
├── App.vue
```

```
├── main.js
└── manifest.json              # 微信小程序 AppID 配置
```

3.3 模块职责说明

文件	负责人	职责
main.py	队友 A	FastAPI 应用入口。定义 POST /api/analyze 和 POST /api/chat 两条路由；/api/analyze 接收图片数组并按 Step 1–5 顺序调用 core/ 各模块；/api/chat 接收 question / context / history 并调用 chat_tool；添加 CORSMiddleware；统一捕获异常并返回标准错误 JSON。
requirements.txt	共同维护	依赖版本锁定。任何人新增依赖后执行 pip freeze > requirements.txt，不手动编辑。
.env	队友 A	存放 QWEN_API_KEY 和 DEEPSEEK_API_KEY。服务器上通过环境变量注入，本地开发各自持有，不提交版本控制。
data/	你	从 LawRefBook/Laws 仓库复制《民法典》合同编和《城市房屋租赁管理办法》Markdown 文件，清洗后保存为纯文本。仅保留租赁相关章节，总量控制在 5 万字以内。
vector_store/	自动生成	ChromaDB 持久化目录，运行 build_index.py 后自动创建。不纳入版本控制，每次部署至新服务器后重新运行 build_index.py 生成。
cache.db	自动创建	SQLite 缓存文件，由 cache_tool.init_db() 首次调用时自动建表。存储 MD5 哈希与 JSON 结果的映射关系。
scripts/build_index.py	你	一次性脚本。读取 data/ 目录文本，使用 bge-small-zh-v1.5 生成向量并写入本地 ChromaDB。执行成功后 vector_store/ 目录出现，无需重复运行。
core/vision_tool.py	你	封装 Qwen-VL-Max API 调用。Prompt 要求返回 {"is_contract": bool, "contract_text": str}。is_contract 为 false 时抛出 ValueError('NOT_CONTRACT')；为 true 时返回该页按条款编号整理的纯文本。main.py 对图片数组逐项调用后，将各页文本顺序拼接得到完整 contract_text。
core/cache_tool.py	你	封装 SQLite 读写操作。init_db() 创建缓存表；get_cache(text) 计算 MD5 并查询历史结果；set_cache(text, result) 写入新结果。全部逻辑约 20 行，无新增依赖。
core/rag_tool.py	你	实现双路检索。路径一：LlamaIndex 向量语义检索 Top-3 法条片段。路径二：预定义关键词表（押金、提前退租、房屋维修等）触发对应民法典条文的强制召回。输入合同文本字符串，输出去重合并后的法条上下文字符串。
core/reviewer.py	你	组装最终 Prompt：合同原文 + 检索法条 + few-shot 示例（取自 DISC-Law-SFT 数据集）+ JSON 格式约束，调用 DeepSeek V3 API。调用 json.loads() 验证响应结构后，将 dict 返回给 main.py。
core/chat_tool.py	你	封装多轮对话 Prompt 构建逻辑。将 context（第一次审查结果 JSON）序列化后注入系统 Prompt，拼接 history 列表构建

文件	负责人	职责
		messages 数组，调用 DeepSeek V3 API 获取自由文本回复。全部逻辑约 30 行，无新增依赖。函数签名：chat(question: str, context: dict, history: list) -> str。
pages/index/index.vue	队友 B	微信小程序审查主页面。调用 uni.chooseImage()（count=10）选取多张图片，逐张经 FileSystemManager 转 base64 后组装数组，通过 uni.request() 发送 POST /api/analyze；解析 analysis_results 数组，按 risk_level 渲染三色风险卡片，并对合同全文中的 original_text 内容进行文本高亮标注；审查完成后提供入口跳转至对话页。
pages/chat/chat.vue	队友 B	微信小程序对话页面。接收来自主页的 context 参数，维护 history 数组；用户输入追问后，将 question / context / history 一并发送至 POST /api/chat，获取回复后将本轮问答追加至 history 并渲染对话气泡。

四、函数签名契约

以下签名须在第 2 周末由你以空壳文件形式交付给队友 A。函数名、参数类型与返回类型锁定，内部实现可暂为 pass 或 raise NotImplementedError。队友 A 据此编写 main.py 路由逻辑，无需等待实现完毕。

```
# core/vision_tool.py
def extract_contract(image_base64: str) -> str:
    """
    单张图片 → 该页合同纯文本。
    非合同图片抛出 ValueError('NOT_CONTRACT')。
    调用方（main.py）负责对图片数组循环调用并拼接结果。
    """
    ...

# core/cache_tool.py
def init_db() -> None: ...
def get_cache(text: str) -> dict | None: ...
def set_cache(text: str, result: dict) -> None: ...

# core/rag_tool.py
def retrieve_legal_context(contract_text: str) -> str:
    """返回合并后的法条上下文字符串。"""
    ...

# core/reviewer.py
def review_contract(contract_text: str, legal_context: str) -> dict:
    """
    返回包含 overall_risk / summary / analysis_results 的 dict。
    调用方保证 json.loads(json.dumps(result)) 可正常执行。
    """
    ...

# core/chat_tool.py
def chat(question: str, context: dict, history: list) -> str:
    """
    基于审查结果 context 和对话历史 history 回答 question。
    返回自由文本字符串，不要求 JSON 格式。
    history 格式：[{'role': 'user'|'assistant', 'content': str}, ...]
    """
    ...
```

五、三人开发分工

5.1 职责总览

	你	队友 A	队友 B
核心职责	AI 核心模块 技术负责人	后端集成 部署运维	前端小程序 技术文书
代码工作量	约 45%	约 35%	约 20%
文书贡献	技术原理章节 （RAG 机制、Prompt 设计）	系统实现章节 （接口设计、部署架构）	全章节统稿 对接其余六位成员
答辩分工	AI 模块技术答疑	演示环境保障 备份方案	PPT 制作 演示主持

5.2 关键交接节点

时间节点	交接内容	方向
第 2 周末	core/ 五个模块函数签名空壳文件（含 chat_tool.py，见第四章）	你 → 队友 A
第 4 周末	POST /api/analyze 及 POST /api/chat 接口文档：请求格式、响应字段说明、Postman 导出文件	队友 A → 队友 B
第 4 周末	JSON 字段约束文档：risk_level 三种取值、risk_type 枚举示例、legal_basis 格式规范、history 数组格式	你 → 队友 B

5.3 并行开发策略

三人在前四周完全并行，互不阻塞。具体措施：

- 队友 B 在第 1–3 周使用硬编码 mockData 调通页面 UI，第 4 周接收接口文档后替换为真实请求，改动范围限于 API 地址与请求参数。
- 队友 A 在收到函数签名空壳后即可完成 main.py 路由逻辑，调用 core/ 各函数时可先以 mock 返回值测试路由完整性。
- 你的 core/ 模块独立测试，每个模块完成后编写单独的 test_*.py 脚本验证输入输出，不依赖 FastAPI 层。

六、引用的开源资源

项目	来源	用途
LawRefBook/Laws	github.com/LawRefBook/Laws	法律知识库数据源。直接复制《民法典》合同编和《城市房屋租赁管理办法》的 Markdown 文件至 data/ 目录。数据持续维护，来源为国家法律法规数据库，属于公有领域，可自由使用。
DISC-Law-SFT 数据集	github.com/FudanDISC/DISC-LawLLM	Prompt few-shot 示例来源。从 DISC-Law-SFT-Pair-QA-released.jsonl 中抽取 3–5 条租赁相关高质量问答对，注入 reviewer.py 的 Prompt 模板以提升输出质量，不部署模型本体。
ChatLaw 双路检索思路	github.com/PKU-YuanGroup/ChatLaw	rag_tool.py 设计参考。不部署 ChatLaw 本身，仅借鉴向量检索与关键词检索双路合并的架构思路，在 rag_tool.py 中实现约 15 行的关键词兜底召回逻辑，无新增依赖。

七、八周开发路线图

周次	你	队友 A	队友 B
第 1 周	申请 Qwen-VL 及 DeepSeek API Key 整理 data/ 法条文本并清洗 运行 build_index.py 验证向量库	立即提交 ICP 备案 购买云服务器及域名 搭建 FastAPI 项目骨架，配置 .env	HBuilderX 新建 UniApp 项目，配置 AppID 完成上传页面框架 以 mockData 调通风险卡片 UI
第 2 周	完成 vision_tool.py（支持单图，多图循环由 main.py 处理） 完成 cache_tool.py 交付函数签名空壳至队友 A	接收函数签名 完成 main.py 路由框架（含 /api/chat 占位） 实现 CORS 中间件与异常处理	根据 JSON 字段约束完善卡片样式 技术文档初稿（系统概述部分）
第 3 周	完成 rag_tool.py（含双路检索） 完成 reviewer.py 初版 完成 chat_tool.py	串联 core/ 各模块 实现多图数组接收与文本拼接逻辑 Postman 联调测试不少于 5 次	调通多图上传 UI 调通对话页面基础框架（使用 mock 数据）
第 4 周	Prompt 调优 接入 few-shot 示例 真实合同照片（多页 ）压测	交付 /api/analyze 及 /api/chat 接口文档至队友 B 接口稳定性测试不少于 10 次	接收 API 文档，替换真实接口 文本高亮功能实现 技术文档初稿完成
第 5 周	边界情况修复 异常输入处理完善	部署至云服务器 Nginx 配置及 SSL 证书申请	技术文档定稿 与其余六位成员对齐商业计划书内容
第 6 周	准备 3 份含霸王条款的真实合同照片 验收演示材料	微信公众平台配置 request 合法域名 体验版小程序真机测试	答辩 PPT 技术章节 演示脚本撰写

周次	你	队友 A	队友 B
第7周	准备 AI 模块深度讲解材料	演示备份方案 服务器稳定性监控	全队 PPT 对齐与彩排
第8周	技术问题答疑准备	答辩当天演示环境保障	答辩主持

八、依赖清单（requirements.txt）

```
fastapi
uvicorn[standard]
python-dotenv
python-multipart
openai                          # 兼容 Qwen-VL 与 DeepSeek 的 SDK 接口格式
llama-index-core
llama-index-vector-stores-chroma
llama-index-embeddings-huggingface
chromadb
sentence-transformers           # bge-small-zh-v1.5 运行依赖
```

九、版本记录

版本	主要变更
v1.0	初始文档：系统简介、技术栈、工作流、目录结构、四周路线图。
v1.1	新增部署红线前置条件；补充 Embedding 模型选型（bge-small-zh-v1.5）。
v1.2	明确 HTTPS 及 ICP 备案为第一周任务；补充 build_index.py 代码片段；新增 cache_tool.py。
v1.3	三项优化分析（拒绝 YOLOv8 方案、采纳 Hash 缓存、LLM 路由内化至 Qwen-VL Prompt）；引入 LawRefBook、DISC-Law-SFT、ChatLaw 三个开源资源。
v1.4	写入三人分工方案；前端改为单文件 H5；新增函数签名契约章节；路线图扩展至八周。
v1.5	前端方案回归微信小程序（UniApp + HBuilderX）；去除非正式表述与 emoji，统一文档风格；补充微信小程序环境限制说明及并行开发策略。
v1.6	新增多图上传支持（vision_tool.py 单图循环，main.py 拼接）；新增多轮对话接口（POST /api/chat，chat_tool.py）；图像标注改为前端文本高亮方案；更新文件目录、函数签名契约、路线图及接口文档说明。

文档版本：v1.6 · 适用阶段：MVP 开发期